TECHNICAL DOCUMENTATION

Krahaso

Price-Comparison Webapp for the Kosovo Market

Version 0.1.0 · July 2026 · Repository: github.com/oltikamberi/Krahaso

1. What We Built

Krahaso (Albanian: "compare") is a price-comparison platform for electronics sold by online retailers in Kosovo. Shoppers can search for a product — a phone, laptop, TV or household appliance — and instantly see which local retailer sells it cheapest, whether it is in stock, and how the price has changed over time.

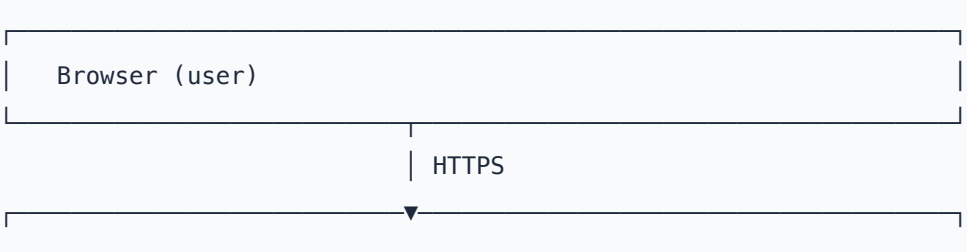
The platform has three core features:

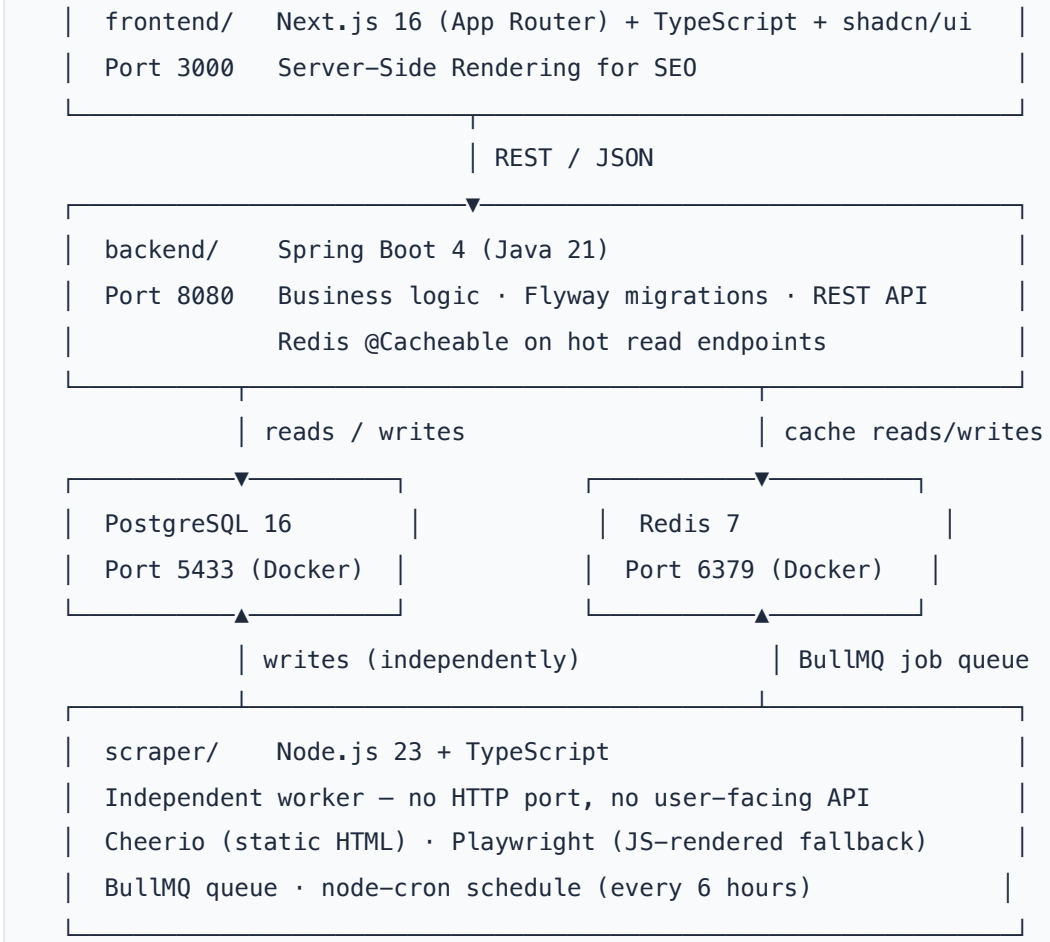
- **Search & browse** — full-text search across all products, and category-based browsing (Phones, Laptops, TVs, Appliances) with price-range filters and sort-by-price.
- **Product detail** — current price, availability, technical specs, and a price-history table showing how the price changed over the last weeks/months.
- **Side-by-side comparison** — select up to 4 products and view them in a comparison table, with the cheapest price highlighted and differing specs bolded so differences are obvious at a glance.

The UI is entirely in Albanian, designed to be simple and mobile-friendly — one big search bar on the home page, a clean card grid for products, and a floating compare bar that follows the user across pages.

2. System Architecture

Krahaso is a **monorepo** containing three independent services that share a single PostgreSQL database. The frontend only ever talks to the backend API; the scraper only ever talks to the database. The database is the single integration point — services are otherwise fully decoupled and can be deployed, restarted, or rewritten independently.





Key architectural principle: the scraper and the backend never call each other. They only share the database. This means a broken scraper never takes down the website, and the website can run fine on seed data before any real scraping is done.

3. Technology Stack & Why We Chose Each

3.1 Frontend — Next.js + TypeScript + shadcn/ui

Technology	Package / Version	Why we chose it
Next.js 16	`next@16`	Server-Side Rendering is essential for a comparison site — product pages must be indexed by Google. A plain React SPA would hurt SEO badly. Next.js App Router gives us SSR/SSG for free.
TypeScript	`typescript@5`	Catches bugs at compile time. All API response types are shared with the frontend so type mismatches are impossible.
Tailwind CSS 4	`tailwindcss@4`	Utility-first CSS makes consistent spacing, colour, and responsive layouts fast to build and easy to maintain.
shadcn/ui	`shadcn@4`	Pre-built accessible components (Card, Table, Badge, Sheet, Select…) so we don't design buttons from scratch. Components are copied as source code — full ownership, no dependency lock-in.
Lucide React	`lucide-react@1`	Consistent icon library used throughout the UI (search, compare, in-stock badges, category icons).

3.2 Backend — Spring Boot + Java 21

Technology	Package / Version	Why we chose it
Spring Boot 4	`spring-boot 4.1`	Opinionated framework that enforces a layered architecture (Controller → Service → Repository). New developers immediately understand where everything lives. 25+ years of enterprise stability.
Java 21	`JDK 21 LTS`	Long-term support release. Virtual threads (Project Loom) for efficient I/O. Strong typing catches bugs the compiler — not production users.
Spring Data JPA + Hibernate	`spring-boot-starter-data-jpa`	ORM that maps Java objects to PostgreSQL tables. Specification API enables dynamic filter queries (category + price range + sort) without hand-writing SQL.
Flyway	`flyway-database-postgresql`	Schema migration tool. The backend owns the database schema — every change is a versioned SQL file. The scraper only reads/writes rows; it never touches schema.

Technology	Package / Version	Why we chose it
Spring Cache + Redis	`spring-boot-starter-data-redis`	Hot endpoints (category list, product detail) are cached in Redis for 5 minutes via @Cacheable annotations. Eliminates repeated identical database queries under load.
Spring Validation	`spring-boot-starter-validation`	Bean Validation (JSR-380) on DTOs prevents bad data entering the system.

3.3 Scraper — Node.js + TypeScript

Technology	Package / Version	Why we chose it
Node.js 23	`node@23`	Node's non-blocking event loop is ideal for scraping — the workload is "wait for network, parse a bit of text." Can run hundreds of concurrent fetches cheaply.
Cheerio	`cheerio@1`	jQuery-like static HTML parser. Used for the default scraping path — fetch raw HTML, extract data. No browser process needed; very fast and lightweight.
got	`got@14`	Modern HTTP client with built-in retries, timeouts, and redirect handling. Used to fetch raw HTML from retailer pages.
Playwright	`playwright@1`	Headless Chromium fallback for JS-rendered pages where product data only appears after JavaScript runs. Reserved for sites that truly need it — 10–50× heavier than Cheerio.
BullMQ	`bullmq@5`	Redis-backed job queue. Each retailer is one job; jobs have automatic retries with exponential backoff if a scrape fails. Ensures resilience — a failing site doesn't block others.
node-cron	`node-cron@4`	Standard cron scheduling. Default schedule: every 6 hours (configurable via SCRAPE_CRON env var).
pg (node-postgres)	`pg@8`	Direct PostgreSQL client. The scraper uses it only to write product rows and price history — it never runs migrations (that is the backend's job).

3.4 Database — PostgreSQL 16

Technology	Why we chose it
PostgreSQL 16	`Relational database for structured data (products belong to categories, have providers, have price history). JSONB columns store variable product specs (a phone has RAM/storage; a TV has screen size) without requiring a schema change per category. Full-text search via tsvector/tsquery for product name search — no separate search engine needed for MVP.`

Technology	Why we chose it
JSONB (specs column)	Scraped products from different sites have wildly different attributes. JSONB lets us store any key-value spec without schema changes, while GIN indexes keep JSONB queries fast.
Generated tsvector column	Full-text search index on product name + brand, generated automatically by Postgres. Powers the /search endpoint without Elasticsearch or any external service.
Redis 7	In-memory store used for two purposes: (1) Spring Boot response cache — avoids repeat database queries for popular pages; (2) BullMQ job queue for the scraper — stores pending/failed scrape jobs with retry state.

4. Database Schema

All migrations live in `backend/src/main/resources/db/migration/` and are owned exclusively by the backend (Flyway). The scraper reads from and writes to these tables but never alters them.

categories

Column	Type	Description
id	BIGSERIAL PK	Auto-incrementing primary key
slug	TEXT UNIQUE	URL-safe identifier, e.g. *telefona*, *laptope*
name	TEXT	Display name in Albanian, e.g. *Telefona*
icon	TEXT	Lucide icon name shown in the category grid

providers

Column	Type	Description
id	BIGSERIAL PK	
slug	TEXT UNIQUE	e.g. *gjirafa50*, *foleja*, *neptun*
name	TEXT	Display name
website_url	TEXT	Root URL of the retailer site
logo_url	TEXT	Optional retailer logo

products

Column	Type	Description
id	BIGSERIAL PK	
category_id	BIGINT FK	References categories.id
provider_id	BIGINT FK	References providers.id
external_id	TEXT	Stable product ID on the retailer's own system
name	TEXT	Product name
brand	TEXT	Manufacturer brand (Apple, Samsung…)
image_url	TEXT	Product image URL from the retailer
product_url	TEXT UNIQUE(provider)	Direct URL to product page — unique per provider
current_price	NUMERIC(10,2)	Latest scraped price

Column	Type	Description
currency	TEXT	EUR by default for all Kosovo retailers
in_stock	BOOLEAN	Current availability
specs	JSONB	Variable specs (RAM, screen size, etc.) — GIN indexed
search_vector	tsvector GENERATED	Auto-maintained full-text search index over name + brand
created_at / updated_at	TIMESTAMPTZ	Auto-managed timestamps

price_history

Column	Type	Description
id	BIGSERIAL PK	
product_id	BIGINT FK	References products.id (CASCADE DELETE)
price	NUMERIC(10,2)	Price at the time of recording
in_stock	BOOLEAN	Availability at the time of recording
scraped_at	TIMESTAMPTZ	When the scraper captured this data point

Price-history change detection: the scraper only writes a new price_history row when the price or availability actually changes. If the scraper runs and finds the same price as last time, it updates the product row silently but writes nothing to price_history. This keeps the table compact and every row meaningful — each row represents a real price event.

5. How the Scraper Works

1. **Schedule triggers** — node-cron fires every 6 hours (configurable). On startup, one cycle also runs immediately so a fresh deployment gets data right away.
2. **Jobs are enqueued** — one BullMQ job per registered retailer adapter is pushed into the Redis queue. Each job carries the provider's slug.
3. **Worker picks up jobs one at a time** — concurrency is limited to 1 (polite to target sites). Failed jobs are retried up to 3 times with exponential backoff (30s, 60s, 120s).
4. **Adapter fetches and normalizes data** — each retailer has its own adapter file (`src/adapters/`). The adapter calls `fetchStatic(url)` to get raw HTML, then prefers JSON-LD structured data (`extractJsonLdProducts()`) over CSS selectors. If the site is JS-rendered, it falls back to `fetchRendered(url)` (Playwright headless browser). Every adapter maps its results to the common `ScrapedProduct` shape.
5. **Database upsert with change detection** — for each product, the database layer checks if a row with the same (provider_id, product_url) already exists. New products are inserted; existing ones are updated. A price_history row is written only if the price or in_stock status actually changed.
6. **Backend serves updated data** — the Next.js frontend fetches from the Spring Boot API, which reads the freshly upserted products. The Redis cache for affected endpoints expires within 5 minutes.

Adapter pattern — adding a new retailer

Adding a new retailer is a **one-file job**:

1. Create `scraper/src/adapters/mysite.ts` implementing the `SiteAdapter` interface (provider info + `fetchProducts()`)
2. Import and add it to the array in `scraper/src/adapters/index.ts`
3. Run `npm run scrape:once` to test it

The queue, scheduling, retries, and database writes all pick it up automatically — no other files need to change.

6. Frontend Pages

Home (/)

One large centered search bar + category tiles (Phones, Laptops, TVs, Appliances). Designed to be the single entry point — one obvious action.

Category (/kategoria/[slug])

Product card grid with price-range filter, sort (newest / price ↑ / price ↓), and in-stock toggle. Filters open in a Sheet drawer on mobile.

Search (/kerko)

Product detail (/produkti/[id])

Full-text search results using PostgreSQL tsvector. Falls back to ILIKE for partial-word matches (e.g. "iph" finds iPhone).

Current price + in-stock badge, specs as key-value pairs, price history table, link to retailer page.

Compare (/krahaso)

Side-by-side table for 2–4 products. Cheapest price highlighted in blue with "Më i liri" badge. Rows with differing values are bolded. Selected via a floating compare bar.

Compare bar (global)

Persistent floating bar at the bottom of every page. Shows selected products, allows removal, navigates to the compare page when ≥2 products are selected. State persisted in localStorage.

7. Backend REST API

Endpoint	Description
GET /api/categories	All categories. Cached in Redis for 5 min.
GET /api/products	Paginated product list. Accepts: category (slug), minPrice, maxPrice, inStock, sort (price_asc / price_desc / newest), page, size.
GET /api/products/search?q=	Full-text + ILIKE search. Returns up to 50 results ordered by price.
GET /api/products/{id}	Single product detail including full specs and complete price history. Cached per product ID.
GET /api/products/compare?ids=1,2,3	Returns 2–4 product details in one call. Used by the compare page.

8. Infrastructure

Both database services run as Docker containers defined in `docker-compose.yml` at the repo root.

Service	Image	Host Port	Notes
PostgreSQL	postgres:16-alpine	5433	Port 5433 (not 5432) to avoid clashing with a local Homebrew Postgres installation
Redis	redis:7-alpine	6379	Standard Redis port. Used by both Spring Boot cache and BullMQ scraper queue

> **Recommended hosting for production (MVP):** Next.js frontend on Vercel (free tier), Spring Boot + scraper on a single small VM (Hetzner €5/mo or Railway/Render free tier), PostgreSQL on Neon or Supabase (free tier up to generous limits). Total infrastructure cost for early-stage: €0–10/month.

9. Growth Path

Now (MVP)

- Monorepo, single VM
- Seed data + demo scraper
- Add real adapters one by one
- Email retailers for permission / feed partnership

Growing (5–10 person team)

- Enable TimescaleDB extension on price_history for time-series queries
- Add Meilisearch/Typesense for instant fuzzy search if catalog grows large
- Split scraper into its own deployment if it needs more resources

- Add user accounts + price-drop alerts

Scaling further

- Spring Boot's modular layout makes extracting a heavy module into its own service straightforward
- Redis already in the stack — ready for rate-limiting, session storage, pub/sub
- Add rotating-proxy support to scraper if retailers implement aggressive bot protection

Business model

- Affiliate links to retailers (revenue share per click/purchase)
- Sponsored placements for retailers who want top position
- Price-drop alert emails (drives return visits)
- Data partnerships with retailers (replaces scraping with official feeds)

10. Repository Structure

```
Krahaso/
├── docker-compose.yml          # PostgreSQL 16 + Redis 7 (Docker)
├── README.md                   # How to run each service
├── .gitignore
│
├── frontend/                   # Next.js 16 App Router
│   ├── src/
│   │   ├── app/                # Pages (layout, page, error)
│   │   │   ├── page.tsx            # Home
│   │   │   ├── kategoria/[slug]/   # Category listing
│   │   │   ├── produkti/[id]/      # Product detail
│   │   │   ├── krahaso/            # Compare page
│   │   │   └── kerko/              # Search results
│   │   ├── components/         # Shared React components
│   │   │   ├── ui/                 # shadcn/ui components
│   │   │   ├── product-card.tsx
│   │   │   ├── compare-bar.tsx
│   │   │   ├── compare-context.tsx
│   │   │   ├── filters-form.tsx
│   │   │   └── site-header.tsx
│   │   └── lib/
│   │       ├── api.ts              # Typed API client (fetch wrappers)
│   │       └── types.ts            # Shared TypeScript interfaces
│   └── package.json
│
├── backend/                    # Spring Boot 4 (Java 21)
│   └── src/main/java/com/krahaso/
│       ├── KrahasoApplication.java
│       ├── config/
│       │   └── WebConfig.java      # CORS configuration
│       ├── category/               # Category module
│       ├── provider/               # Provider module
│       ├── product/                # Product module (main business logic)
│       │   └── dto/                # Request/response DTOs
│       └── price/                  # Price history module
│   └── src/main/resources/
│       ├── application.properties
│       └── db/migration/
│           ├── V1__init.sql        # Schema (tables, indexes)
│           └── V2__seed.sql        # 11 sample products
│
└── scraper/                    # Node.js 23 worker
    └── src/
```

```
├── index.ts              # Scheduled mode entry point
├── run-once.ts           # Single-cycle test runner
├── config.ts             # Environment variables
├── types.ts              # SiteAdapter interface + ScrapedProduct
├── http.ts               # fetchStatic, fetchRendered, extractJsonLdProducts
├── db.ts                 # ensureProvider, upsertProducts (change detection)
├── scrape.ts             # runAdapter orchestrator
├── queue.ts              # BullMQ queue + worker
└── adapters/
    ├── index.ts          # Registry of active adapters
    ├── demo.ts           # Fully working offline demo adapter
    └── gjirafa50.ts      # Skeleton (disabled — verify selectors first)
```